

Furkan Büyüksarıkulak

ID 22002097

EE-102, Section 003

Lab 6 : Greatest Common Divisor

Experiment 6

15.11.2023

AIM:

This lab project aimed to design a circuit that gives greatest common division of two 8-bit numbers as output. Moreover, 16 pins were used for two 8-bit numbers, the results were displayed via LEDs, and buttons were assigned to sets and resets.

METHODOLOGY:

In this circuit design, the Euclidean algorithm was used to find the GCD: To find the greatest common divisor of the numbers, we first compare the numbers, if the numbers are equal to each other, the greatest common divisor is one of the numbers, if not, we subtract from each other, we repeat this until the numbers are equal to each other, and When equal, one of the numbers is gcd. Our module is designed as an FSM which is a Moore Machine meaning the output depends on the current state. Using combinational circuit is more expensive and much harder to make and also FSM is slower. Then, in our simulation it took 15 clock cycles after lifting the load signal to calculate GCD of 140 and 12. It is optimized enough, but maybe it could be more optimized by editing the clock cycle and also combinational circuits would yield maximum optimization. After that, we implemented the described Euclidean algorithm with Moore Machine. Then testbench was run for the input numbers 140 and 12. Finally, thanks to the constraint file, the pins and LEDs representing the inputs and outputs were determined for Basys3 and also assigned to the set and reset buttons.

DESIGN SPECIFICATIONS:

Our design has 5 inputs and 1 output :

IN	n_a	8 bit	Takes in the first number
IN	n_b	8 bit	Takes in the second number
IN	clk	1 bit	100 Mhz base clock of Basys3
IN	reset	1 bit	Reset the registers in the GCD calculator
IN	load	1 bit	Load the registers and strat the GCD operation
OUT	gcd_out	8 bit	Outputs the GDC of n_a and n_b

Table1 : In and out's of design

We use 3 submodules to modulate our design.

Firstly, full adder submodule takes 1 bit inputs and outputs the sum and carry of this calculation. We used this module, whose diagram is shown in Figure 1, to create the subtraction module.

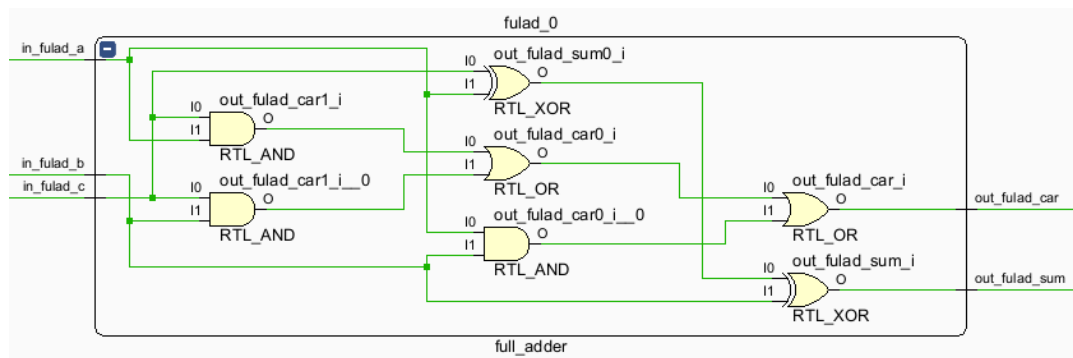


Figure 1 : Full Adder Schematic

Secondly, subtraction module takes two 8 bit binary inputs (in_a and in_b), then it subtracts in_a from in_b and gives the result as out_1. It does ripple carry subtraction by using 8 full adders. Its schematic can be observed in Figure 2.

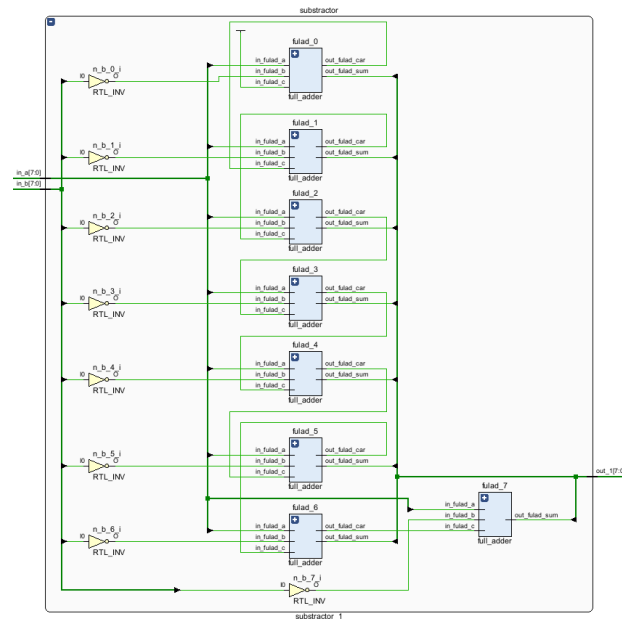


Figure 2 : Subtractor Schematic

Thirdly, comparator submodule takes two 8 bit binary inputs (in_a and in_b) and this module finds the relation between these inputs. The submodule has 3 outputs which are gre, equ and less. If the inputs are equal, equ is 1 and others are 0. If in_a is greater than in_b, gre is 1 and others are 0. If in_b is greater than in_a, less is 1 and others are 0. The schematic of the module is as seen in Figure 3.

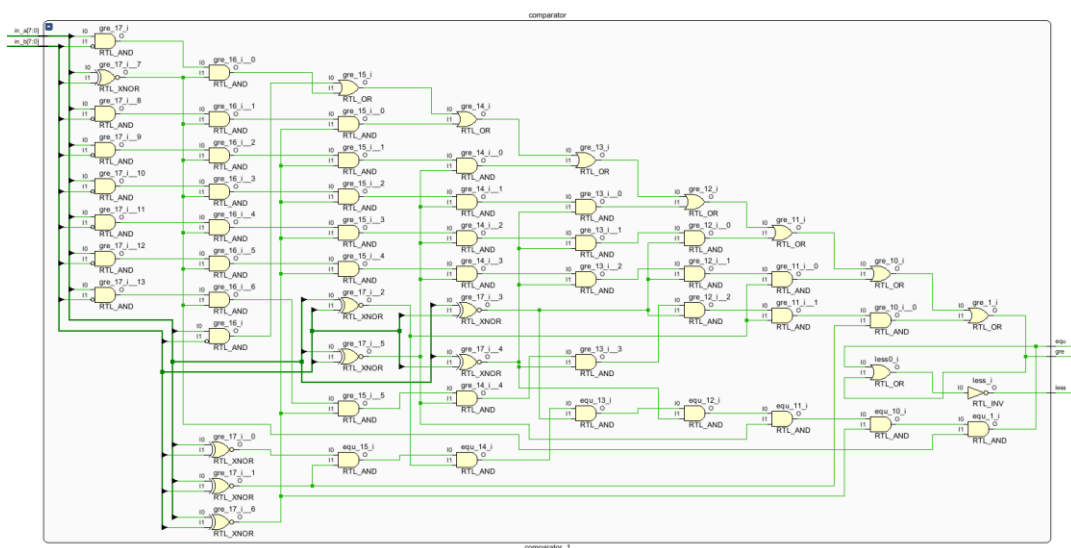


Figure 3 : Comparator Schematic

Finally, our main module , created as a moore machine, contains registers. Euclidean was used to calculate the gcd value of our first loads.

Thanks to the comparator submodule, the output values of each register are compared instantly. If the result of the first input is greater than the second, the second is subtracted from the first. If the result of the second is greater than the first, the place is swapped. These processes continue until the results are equal. If the results are equal, the common divisor is one of the numbers. The schematic of the main module is as shown in Figure 4.

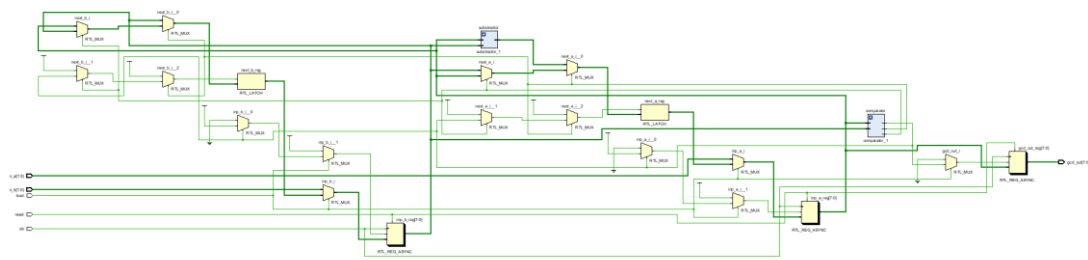


Figure 4 : GCD Schematic

RESULTS:

We wrote a testbench code to check whether the code we wrote worked as expected. We entered "00001100" and "10001100" as input. While the load signal is 1 at the beginning, it is changed to 0 afterwards. While the reset is 0 at the beginning, it is made 1 at 550ns. The outputs of the registers can be observed on the rising edges of the clock. The design works as expected and correctly, as observed in Figure 5 and Figure 6.

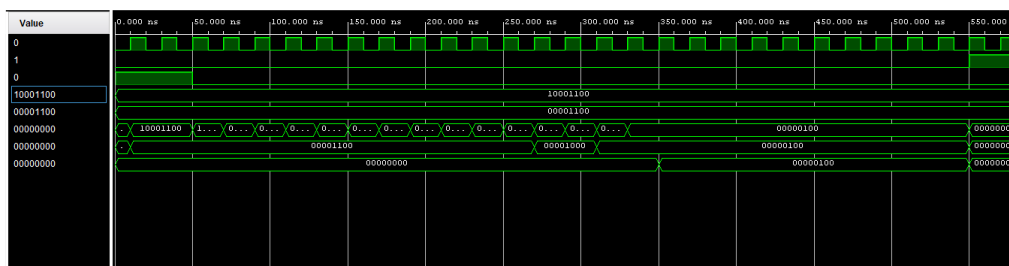


Figure 5 : Testbench Simulation (1)

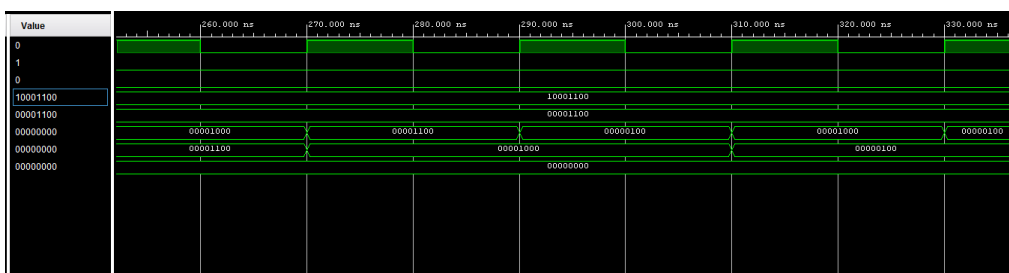


Figure 6 : Testbench Simulation(2)

The pin configurations assigned on Basys 3 are as follows.

n_a	Switches from V2 to R2 (Left)
n_b	Switches from V17 to W13 (Right)
reset	BTND (U17)
load	BTNU (T18)
gcd_out	LEDs from U16 to V14 (Right most LEDs)

Table 2 : Pin Configurations

Some example cases are shown below.

CASE1:

in/out	Binary	Decimal
n_a	111100	60
n_b	11000	24
gcd_out	1100	12
Expected out	1100	12
Reset = "1"	00000000	0

Table 3 : Case 1

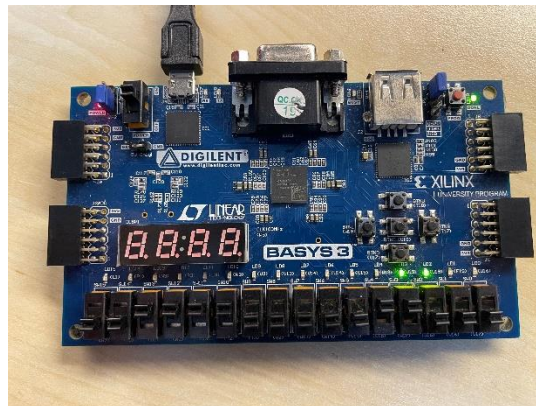


Figure 7 : Case 1

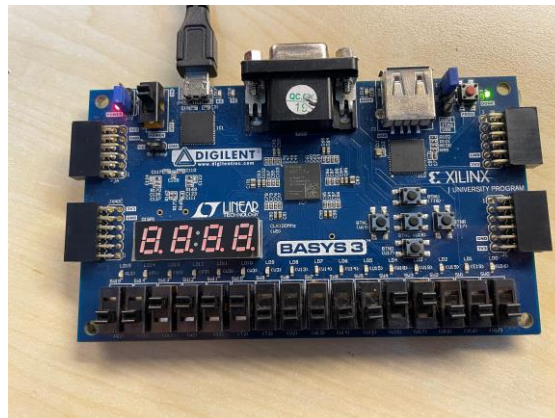


Figure 8: Reset = "1" Case

CASE2:

in/out	Binary	Decimal
n_a	111000	56
n_b	100000	32
gcd_out	1000	8
Expected out	1000	8

Table 4 : Case 2

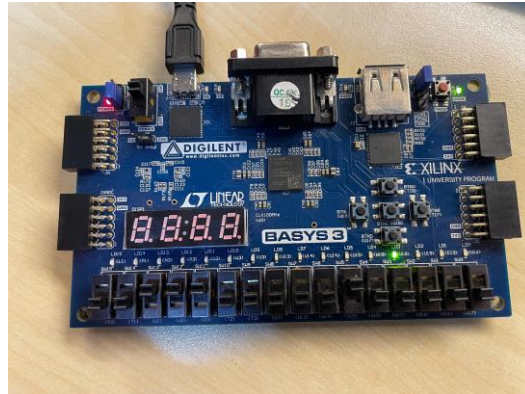


Figure 9 : Case 2

CASE3:

in/out	Binary	Decimal
N_a	101111	47
N_b	100011	35
Gcd_out	1	1
Expected out	1	1

Table 5 : Case 3

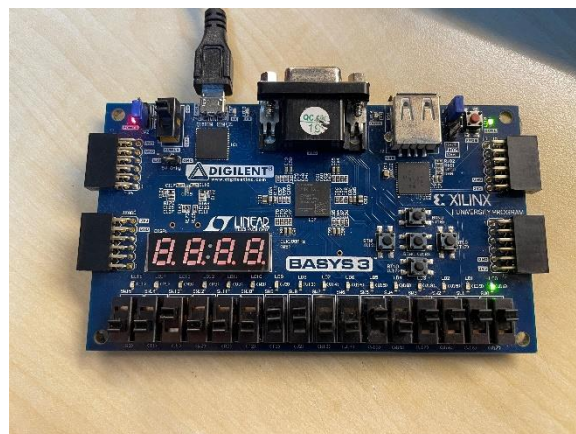


Figure 10 : Case 3

As can be seen that our code works as intended on our Basys3 board.

CONCLUSION:

In this laboratory, the aim was to find the greatest common divisor of two 8-bit binary numbers. We designed the Moore machine that performs this operation using the Euclidean algorithm. Our design was able to fulfill the intended task and worked successfully. We reached this conclusion through our observations in simulation and experiment. In this experiment we learned how to use registers in VHDL design. We also learned to design with FSM machines. No errors or problems were encountered during the design process, but it did take time to understand and establish the logic of the design.

APPENDIX:

main_gcd.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity main_gcd is
    Port (n_a, n_b : in std_logic_vector(7 downto 0);
    clk, load, reset : in std_logic;
    gcd_out : out std_logic_vector(7 downto 0):= (others => '0')
    );
end main_gcd;

architecture Behavioral of main_gcd is

    component comparator_1 is
        Port (
            in_a : in std_logic_vector(7 downto 0 );
            in_b : in std_logic_vector(7 downto 0 );
            gre : out std_logic;
            less : out std_logic;
            equ : out std_logic);
        end component;

    component subtractor_1 is
        Port (
            in_a : in std_logic_vector(7 downto 0);
            in_b : in std_logic_vector(7 downto 0);
            out_1 : out std_logic_vector(7 downto 0));
        end component;

    signal inp_a, inp_b : std_logic_vector(7 downto 0):=(others=> '0');
    signal next_a, next_b : std_logic_vector(7 downto 0);
    signal great, eq, les : std_logic;
    signal sut : std_logic_vector(7 downto 0);

begin
```

comparator : comparator_1

Port Map(in_a => inp_a,

in_b => inp_b,

gre => great,

equ => eq,

less => les);

subtractor: subtractor_1

Port Map(in_a => inp_a,

in_b => inp_b,

out_1 => sut);

process (clk, reset) begin

if reset = '1' then

inp_a <= (others => '0');

inp_b <= (others => '0');

gcd_out <= (others => '0');

elsif rising_edge(clk) then

if load = '1' then

inp_a <= n_a;

inp_b <= n_b;

elsif eq = '1' then

gcd_out <= inp_a;

else

inp_a <= next_a;

inp_b <= next_b;

end if;

end if;

end process;

process (inp_a,inp_b,sut,eq,les,great) begin

if great = '1' then

next_b <= inp_b;

next_a <= sut;

elsif les = '1' then

next_a <= inp_b;

next_b <= inp_a;

elsif eq = '1' then

```
    next_a <= inp_a;

    next_b <= inp_b;

end if;

end process;
```

```
end Behavioral;
```

comparator_1.vhd

```
library IEEE;
```

```
use IEEE.STD_LOGIC_1164.ALL;
```

```
entity comparator_1 is
```

```
Port (
```

```
    in_a : in std_logic_vector(7 downto 0 );
```

```
    in_b : in std_logic_vector(7 downto 0 );
```

```
    gre : out std_logic;
```

```
    less : out std_logic;
```

```
    equ : out std_logic);
```

```
end comparator_1;
```

```
architecture Behavioral of comparator_1 is
```

```
    signal equal : std_logic_vector(7 downto 0);
```

```
    signal greater : std_logic_vector(7 downto 0);
```

```
    signal equ_1, gre_1 : std_logic;
```

```
begin
```

```
    process(in_a, in_b)
```

```
    begin
```

```
        for n in 0 to 7 loop
```

```
            equal(n) <= in_a(n) xnor in_b(n);
```

```
        end loop;
```

```
    end process;
```

```
    equ_1 <= equal(0) and equal(1) and equal(2) and equal(3) and equal(4) and equal(5) and equal(6) and equal(7);
```

```

gre_1 <=(in_a(7) and (not in_b(7))) or
      (in_a(6) and (not in_b(6)) and equal(7)) or
      (in_a(5) and (not in_b(5)) and equal(7) and equal(6)) or
      (in_a(4) and (not in_b(4)) and equal(7) and equal(6) and equal(5)) or
      (in_a(3) and (not in_b(3)) and equal(7) and equal(6) and equal(5) and equal(4)) or
      (in_a(2) and (not in_b(2)) and equal(7) and equal(6) and equal(5) and equal(4) and equal(3)) or
      (in_a(1) and (not in_b(1)) and equal(7) and equal(6) and equal(5) and equal(4) and equal(3)and equal(2)) or
      (in_a(0) and (not in_b(0)) and equal(7) and equal(6) and equal(5) and equal(4) and equal(3)and equal(2) and equal(1));

```

```
less <= (equ_1 nor gre_1);
```

```
gre <= gre_1;
```

```
equ <= equ_1;
```

```
end Behavioral;
```

full_adder.vhd

```
library IEEE;
```

```
use IEEE.STD_LOGIC_1164.ALL;
```

```
entity full_adder is
```

```
Port(
```

```
in_fulad_c : in std_logic;
```

```
in_fulad_a : in std_logic ;
```

```
in_fulad_b : in std_logic ;
```

```
out_fulad_sum : out std_logic ;
```

```
out_fulad_car : out std_logic );
```

```
end full_adder;
```

```
architecture Behavioral of full_adder is
```

```
begin
```

```
out_fulad_sum <= in_fulad_c xor in_fulad_a xor in_fulad_b;
```

```
out_fulad_car <= (in_fulad_c and in_fulad_a) or (in_fulad_c and in_fulad_b) or (in_fulad_a and in_fulad_b);
```

```
end Behavioral;
```

subtractor_1.vhd

```
library IEEE;
```

```
use IEEE.STD_LOGIC_1164.ALL;
```

```
entity subtractor_1 is
```

```
Port (
```

```
in_a : in std_logic_vector(7 downto 0);
```

```
in_b : in std_logic_vector(7 downto 0);
```

```
out_1 : out std_logic_vector(7 downto 0));
```

```
end subtractor_1;
```

```
architecture Behavioral of subtractor_1 is
```

```
component full_adder is
```

```
Port (
```

```
in_fulad_c : in std_logic;
```

```
in_fulad_a : in std_logic ;
```

```
in_fulad_b : in std_logic ;
```

```
out_fulad_sum : out std_logic ;
```

```
out_fulad_car : out std_logic );
```

```
end component;
```

```
signal c_n : std_logic_vector(7 downto 0);
```

```
signal n_b : std_logic_vector(7 downto 0);
```

```
begin
```

```
n_b(0) <= not in_b(0);
```

```
n_b(1) <= not in_b(1);
```

```
n_b(2) <= not in_b(2);
```

```
n_b(3) <= not in_b(3);
```

```
n_b(4) <= not in_b(4);
```

```
n_b(5) <= not in_b(5);
```

```
n_b(6) <= not in_b(6);
```

```
n_b(7) <= not in_b(7);
```

```
fulad_0 : full_adder
```

```
Port Map(in_fulad_a => in_a(0),  
         in_fulad_b  => n_b(0),  
         in_fulad_c  => '1',  
         out_fulad_car => c_n(0),  
         out_fulad_sum => out_1(0));
```

```
fulad_1 : full_adder
```

```
Port Map(in_fulad_a => in_a(1),  
         in_fulad_b  => n_b(1),  
         in_fulad_c  => c_n(0),  
         out_fulad_car => c_n(01),  
         out_fulad_sum => out_1(1));
```

```
fulad_2 : full_adder
```

```
Port Map(in_fulad_a => in_a(2),  
         in_fulad_b  => n_b(2),  
         in_fulad_c  => c_n(1),  
         out_fulad_car => c_n(2),  
         out_fulad_sum => out_1(2));
```

```
fulad_3 : full_adder
```

```
Port Map(in_fulad_a => in_a(3),  
         in_fulad_b  => n_b(3),  
         in_fulad_c  => c_n(2),  
         out_fulad_car => c_n(3),  
         out_fulad_sum => out_1(3));
```

```
fulad_4 : full_adder
```

```
Port Map(in_fulad_a => in_a(4),  
         in_fulad_b  => n_b(4),  
         in_fulad_c  => c_n(3),  
         out_fulad_car => c_n(4),  
         out_fulad_sum => out_1(4));
```

```
fulad_5 : full_adder
```

```
Port Map(in_fulad_a => in_a(5),  
         in_fulad_b  => n_b(5),  
         in_fulad_c  => c_n(4),  
         out_fulad_car => c_n(5),
```

```

        out_fulad_sum => out_1(5));

fulad_6 : full_adder

Port Map(in_fulad_a => in_a(6),

        in_fulad_b   => n_b(6),

        in_fulad_c   => c_n(5),

        out_fulad_car => c_n(6),

        out_fulad_sum => out_1(6));

fulad_7 : full_adder

Port Map(in_fulad_a => in_a(7),

        in_fulad_b   => n_b(7),

        in_fulad_c   => c_n(6),

        out_fulad_car => c_n(7),

        out_fulad_sum => out_1(7));

end Behavioral;

```

testbench_gdc.vhd

```

library IEEE;

use IEEE.STD_LOGIC_1164.ALL;


entity testbench_gdc is
end testbench_gdc;


architecture Behavioral of testbench_gdc is


    signal clk, reset, load : std_logic;

    signal in_a, in_b, gcd_out : std_logic_vector(7 downto 0);


    component main_gcd is

        Port (n_a, n_b: in std_logic_vector(7 downto 0);

            clk, load, reset : in std_logic;

            gcd_out : out std_logic_vector(7 downto 0):= (others => '0')

        );

    end component;

end component;

```

```

begin

test: main_gcd

Port Map(n_a => in_a,

n_b => in_b,

clk => clk,

load => load,

reset => reset,

gcd_out => gcd_out

);

clock: process begin

clk <= '0';

wait for 10ns;

clk <= '1';

wait for 10 ns;

end process;

stimulus: process begin

load <= '1';

in_a <= "10001100";

in_b <= "00001100";

reset<= '0';

wait for 50ns;

load <= '0';

wait for 500ns;

reset<= '1';

wait;

end process;

end Behavioral;

```

cons_gdc.xdc

```

set_property PACKAGE_PIN V17 [get_ports {n_b[0]}]

set_property IOSTANDARD LVCMOS33 [get_ports {n_b[0]}]

set_property PACKAGE_PIN V16 [get_ports {n_b[1]}]

set_property IOSTANDARD LVCMOS33 [get_ports {n_b[1]}]

```



```
set_property PACKAGE_PIN W16 [get_ports {n_b[2]]}

set_property IOSTANDARD LVCMOS33 [get_ports {n_b[2]]}

set_property PACKAGE_PIN W17 [get_ports {n_b[3]]}

set_property IOSTANDARD LVCMOS33 [get_ports {n_b[3]]}

set_property PACKAGE_PIN W15 [get_ports {n_b[4]]}

set_property IOSTANDARD LVCMOS33 [get_ports {n_b[4]]}

set_property PACKAGE_PIN V15 [get_ports {n_b[5]]}

set_property IOSTANDARD LVCMOS33 [get_ports {n_b[5]]}

set_property PACKAGE_PIN W14 [get_ports {n_b[6]]}

set_property IOSTANDARD LVCMOS33 [get_ports {n_b[6]]}

set_property PACKAGE_PIN W13 [get_ports {n_b[7]]}

set_property IOSTANDARD LVCMOS33 [get_ports {n_b[7]]}

set_property PACKAGE_PIN V2 [get_ports {n_a[0]]}

set_property IOSTANDARD LVCMOS33 [get_ports {n_a[0]]}

set_property PACKAGE_PIN T3 [get_ports {n_a[1]]}

set_property IOSTANDARD LVCMOS33 [get_ports {n_a[1]]}

set_property PACKAGE_PIN T2 [get_ports {n_a[2]]}

set_property IOSTANDARD LVCMOS33 [get_ports {n_a[2]]}

set_property PACKAGE_PIN R3 [get_ports {n_a[3]]}

set_property IOSTANDARD LVCMOS33 [get_ports {n_a[3]]}

set_property PACKAGE_PIN W2 [get_ports {n_a[4]]}

set_property IOSTANDARD LVCMOS33 [get_ports {n_a[4]]}

set_property PACKAGE_PIN U1 [get_ports {n_a[5]]}

set_property IOSTANDARD LVCMOS33 [get_ports {n_a[5]]}

set_property PACKAGE_PIN T1 [get_ports {n_a[6]]}

set_property IOSTANDARD LVCMOS33 [get_ports {n_a[6]]}

set_property PACKAGE_PIN R2 [get_ports {n_a[7]]}

set_property IOSTANDARD LVCMOS33 [get_ports {n_a[7]]}


# LEDs

set_property PACKAGE_PIN U16 [get_ports {gcd_out[0]]}

set_property IOSTANDARD LVCMOS33 [get_ports {gcd_out[0]]}

set_property PACKAGE_PIN E19 [get_ports {gcd_out[1]]}

set_property IOSTANDARD LVCMOS33 [get_ports {gcd_out[1]]}

set_property PACKAGE_PIN U19 [get_ports {gcd_out[2]]}

set_property IOSTANDARD LVCMOS33 [get_ports {gcd_out[2]]}

set_property PACKAGE_PIN V19 [get_ports {gcd_out[3]]}

set_property IOSTANDARD LVCMOS33 [get_ports {gcd_out[3]]}
```

```
set_property PACKAGE_PIN W18 [get_ports {gcd_out[4]]}
set_property IOSTANDARD LVCMOS33 [get_ports {gcd_out[4]]}
set_property PACKAGE_PIN U15 [get_ports {gcd_out[5]]}
set_property IOSTANDARD LVCMOS33 [get_ports {gcd_out[5]]}
set_property PACKAGE_PIN U14 [get_ports {gcd_out[6]]}
set_property IOSTANDARD LVCMOS33 [get_ports {gcd_out[6]]}
set_property PACKAGE_PIN V14 [get_ports {gcd_out[7]]}
set_property IOSTANDARD LVCMOS33 [get_ports {gcd_out[7]]}
```

Clock signal

```
set_property PACKAGE_PIN W5 [get_ports clk]
set_property IOSTANDARD LVCMOS33 [get_ports clk]
```

```
set_property PACKAGE_PIN T18 [get_ports load]
set_property IOSTANDARD LVCMOS33 [get_ports load]
set_property PACKAGE_PIN U17 [get_ports reset]
set_property IOSTANDARD LVCMOS33 [get_ports reset]
```